

Accessera enskilda tecken

Man kan enkelt accessera enskilda tecken i en `string`. Ofta vill man t.ex. iterera över tecken och söka efter något visst tecken eller utföra någon annan operation som kräver att man direkt kommer åt de enskilda tecknen. Klassen `string` har två olika metoder för att utföra detta. Det första sättet använder sig av metoden `at()` som returnerar en referens till ett tecken på ett givet index. Ett exempel på att använda `at()`:

```
#include <iostream>
#include <string>

int main () {
    string S1 = "Hello world";

    // ändra 'w' till 'W'
    S1 [6] = 'W';

    cout << S1 << endl;
}
```

Samma exemempel men med `at()` ser ut på följande sätt:

```
#include <iostream>
#include <string>

int main () {
    string S1 = "Hello world";

    // ändra 'w' till 'W'
    S1.at (6) = 'W';

    cout << S1 << endl;
}
```

De två olika metoderna har en fundamental skillnad. Metoden `at()` kontrollerar om det givna indexet är giltigt, d.v.s. att det ligger inom intervallet 0 till `length() - 1`. Strängar indexeras precis som normala vektorer från 0. Ett försök att accessera ett tecken förbi slutet av strängen leder då man använder `at()` till att en *exception* `out_of_range` kastas. Se [Kapitel 20](#) för mera information om exceptions. Använder man `[]` för att indexera strängen görs ingen dylik kontroll och diverse fel kan uppstå av att accessera förbi slutet på en sträng. Om man dock är noggrann med att inte accessera förbi slutet med `[]` är det ingen skillnad på funktionaliteten på de två sätten.